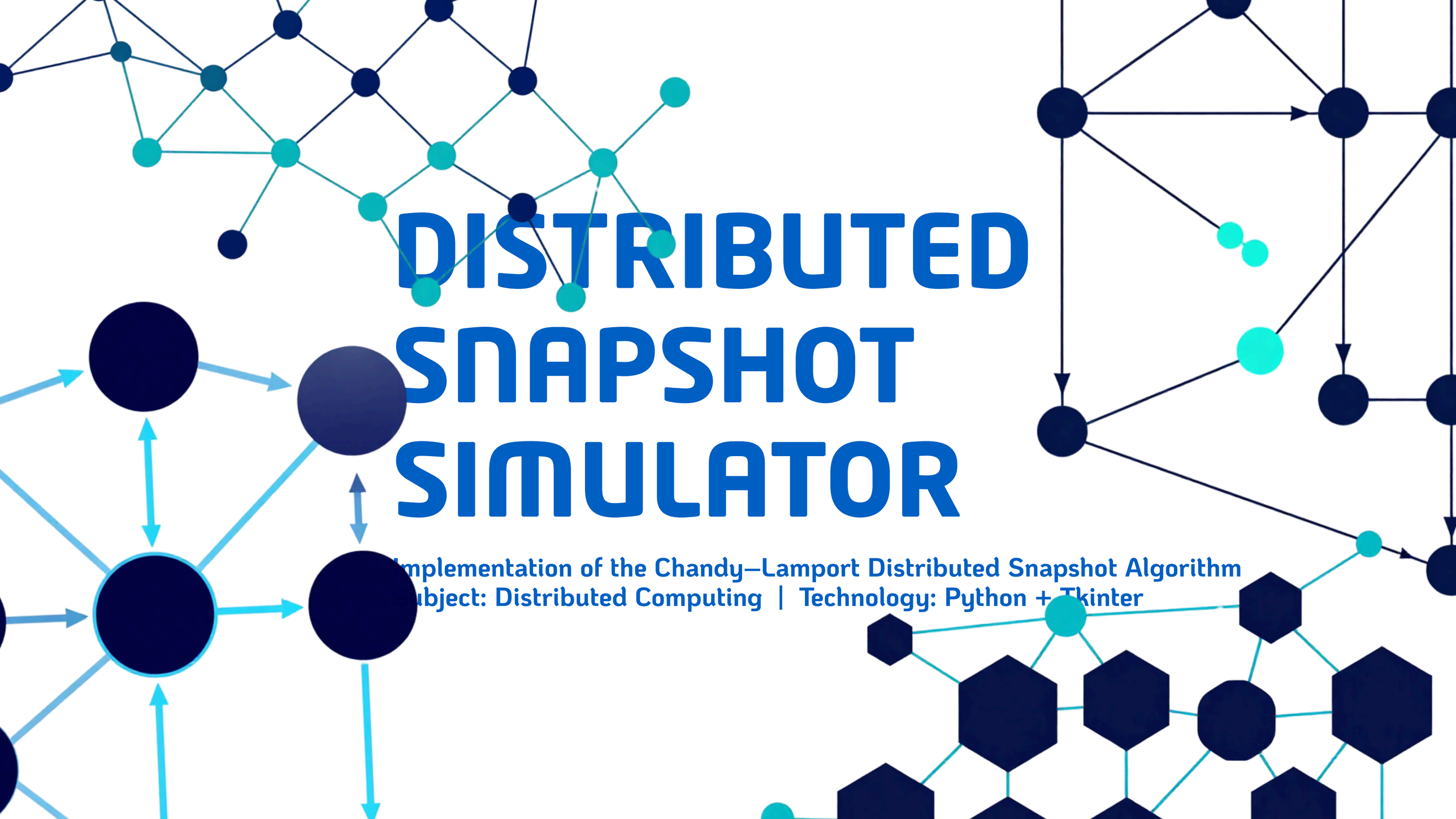


DISTRIBUTED SNAPSHOT SIMULATOR

Implementation of the Chandy–Lamport Distributed Snapshot Algorithm

Subject: Distributed Computing | Technology: Python + Tkinter

DISTRIBUTED SNAPSHOT
SIMULATOR

PROJECT TEAM

Distributed Computing | Python + Tkinter

Jibin Thomas
26PG110241

Nandani Gohil
26PG110230

Vishalbhai Makwana
26PG110243

MCA-AI MINI PROJECT

PROJECT OVERVIEW

Distributed systems consist of multiple concurrent processes running across networked nodes — with no shared memory and no global clock. Capturing a consistent global state in such an environment is a fundamental challenge.

In-transit messages, asynchronous communication, and independent process states make it impossible to simply "pause" the system and take a snapshot.

The Chandy–Lamport Algorithm solves this by enabling processes to record their own local states and the states of communication channels — without halting execution.

Project Goal: Simulate and visualize the Chandy–Lamport distributed snapshot algorithm using Python and Tkinter — demonstrating how a consistent global state is captured across distributed processes in real time.

ALGORITHM OVERVIEW

The Chandy-Lamport Algorithm is a classic distributed systems protocol designed to capture a consistent global snapshot of a running distributed system. It records the state of each process and every communication channel without halting execution, enabling fault tolerance, deadlock detection, and system debugging across distributed nodes.

Core Mechanism: A process initiates the snapshot by saving its local state and broadcasting a special Marker message along all outgoing channels. Each receiving process records its own state upon first receiving a Marker, then records all in-transit messages on incoming channels until a Marker arrives. Together, these form a consistent global snapshot.

SIMULATOR COMPONENTS

Process & Node Management
Create and manage distributed process nodes. Each node runs independently, maintains local state, and communicates via directed channels — simulating a real distributed system environment.

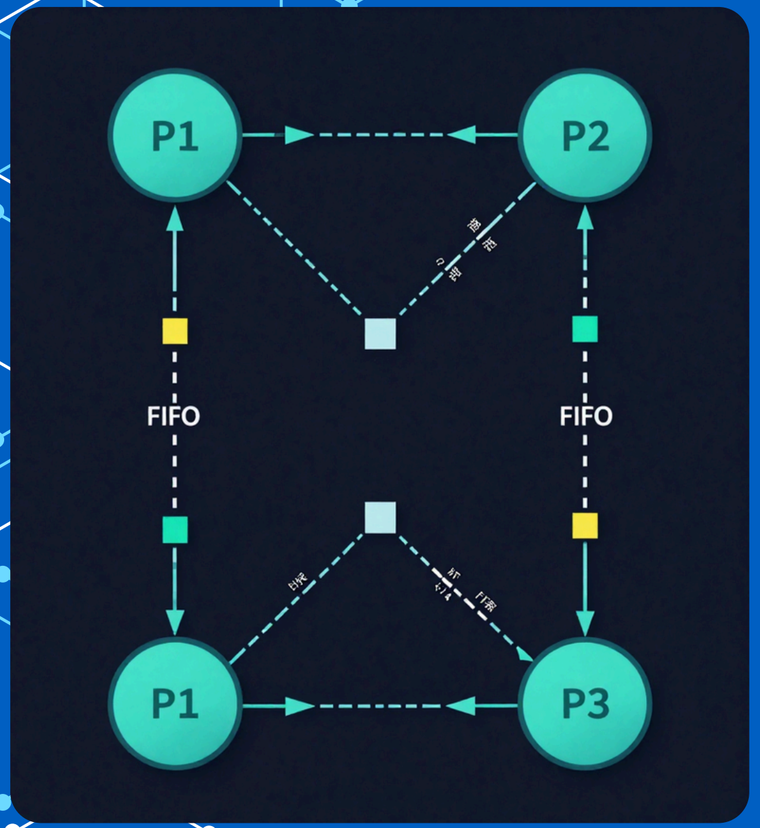
Message Passing & Channel Visualization
Visualise real-time message flow across channels using Tkinter. Directed edges represent communication links, and in-transit messages are tracked during snapshot recording.

Snapshot Recording & Global State
Capture consistent global snapshots using Chandy-Lamport markers. Recorded process states and channel states are displayed clearly for analysis and verification.

IMPLEMENTATION

Language & GUI: Python 3 with Tkinter for interactive process visualization
Process Network: Nodes represented as canvas widgets with live state display
Event Handling: Click-driven message sending and snapshot triggers
Message Queues: Per-channel FIFO queues simulate inter-process communication
Marker Propagation: Chandy-Lamport markers forwarded on all outgoing channels
Snapshot Storage: Local state + in-transit messages captured per process





SYSTEM MODEL

The simulated system consists of multiple logical processes interconnected by directed communication channels. Each process maintains its own local state and communicates exclusively via message passing.

- Processes: Independent nodes with internal state
- Channels: Directed FIFO links between processes
- Application Messages: Regular data exchanged between nodes
- Marker Messages: Special control signals used to capture consistent global snapshots
- Each process records its state upon receiving the first marker

SNAPSHOT EXECUTION FLOW

1

Step 1: Initiate Snapshot
The initiating process records its own local state immediately. It saves its current variable values, message queues, and execution context before any further computation proceeds.

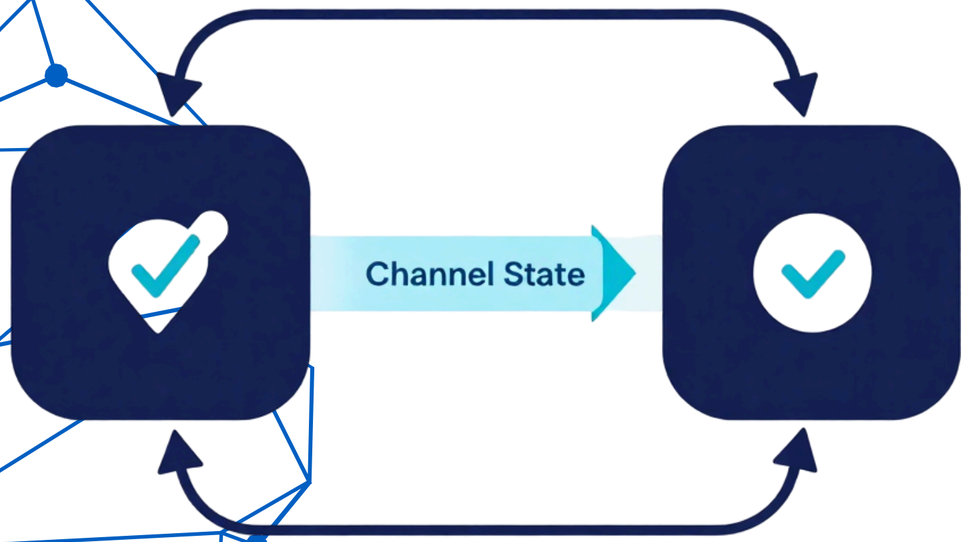
2

Step 2: Broadcast Markers
The initiator sends a MARKER token on every outgoing channel. Each receiving process records its local state upon first marker receipt and begins logging all subsequent incoming messages on each channel until a marker arrives on that channel.

3

Step 3: Collect Global Snapshot
Once all processes have received markers on every incoming channel, local states and channel states are aggregated. The union of all recorded states forms a consistent global snapshot of the distributed system.

CORRECTNESS PRINCIPLES



Consistent Global Cut: Every process records its local state at a well-defined point, ensuring all processes agree on a unified snapshot. No process captures a state that contradicts another — the cut is globally consistent across the entire distributed system.

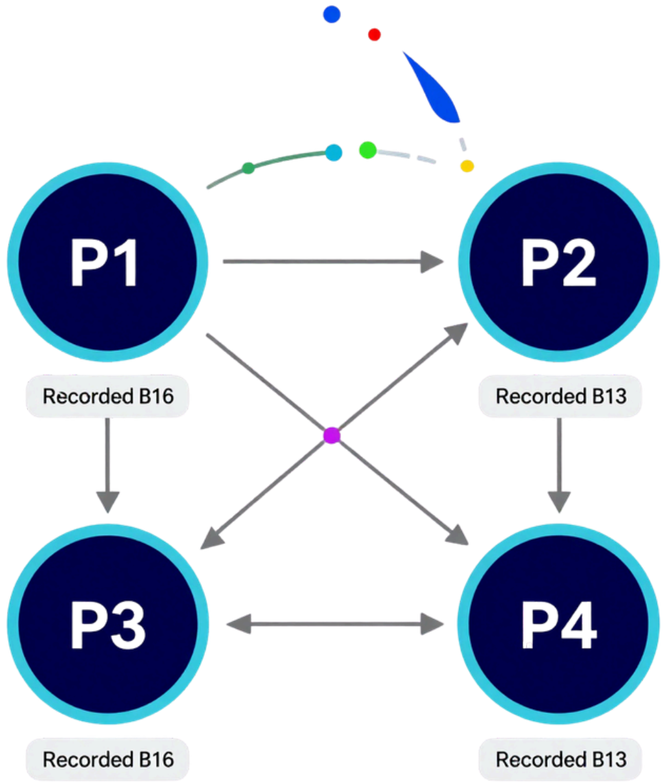
In-Transit Message Capture: Messages sent before a process records its state but received after are captured in channel state. This guarantees zero duplication and zero loss — every in-flight message is accounted for exactly once in the final global snapshot.

SIMULATOR RESULTS & VISUALIZATION

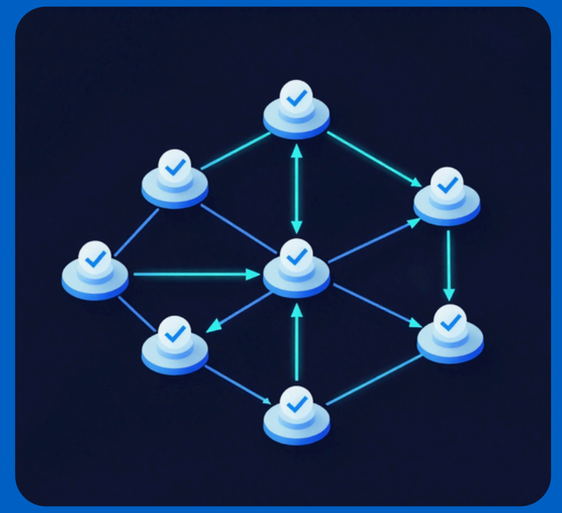
Tkinter GUI displays live process states, channel queues, and recorded snapshots in real time.

- Nodes shown as labeled circles with state values
- Directed arrows represent communication channels
- Marker propagation animated step by step
- Snapshot panel records each process state on marker receipt
- Students observe distributed consistency without shared memory

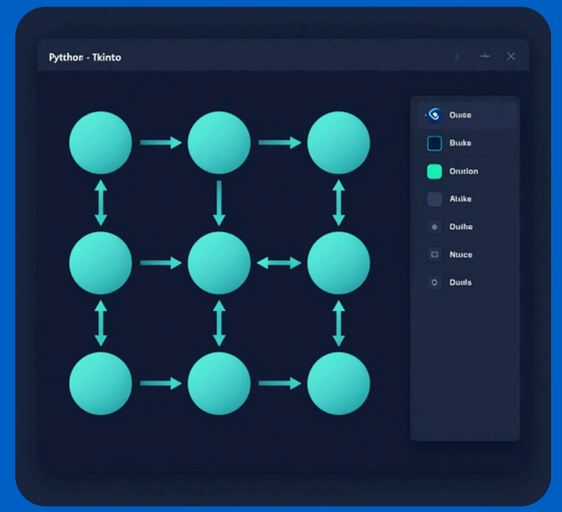
Educational Value: Bridges theory and practice — learners visualize why global state cannot be captured at a single instant across distributed processes.



CONCLUSION & FUTURE SCOPE



- Algorithm successfully simulated using Chandy-Lamport protocol with accurate global state capture across distributed processes in a Python + Tkinter GUI environment.
- Global state made visually understandable through real-time node diagrams, channel state tracking, and interactive snapshot triggers — ideal for academic learning.
- Future Scope: Add pause/resume controls, support larger dynamic networks, integrate event logs, enable snapshot export (JSON/CSV), and extend to fault-tolerant simulation.



THANK YOU

Questions & Discussion | DISTRIBUTED SNAPSHOT SIMULATOR
Distributed Computing — MCA-AI Mini Project